

Sockets and Linux network programming

Mobile and Sensor Networks – A.Y. 2025/26

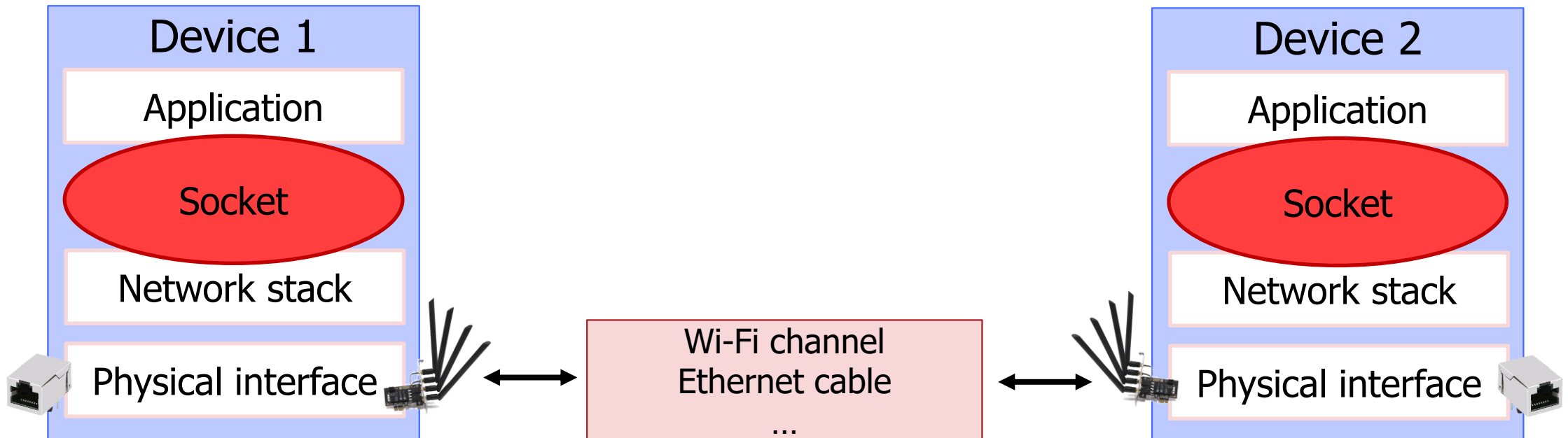
Francesco Raviglione

Linux network programming in Python

- This lab activity will teach you how to write **Python code** to send/receive packets over a network, using **Linux**
- Very important **skill** to develop when working with mobile and sensor networks
- The focus will be on a **Wi-Fi network** with two laptops connected to the same AP
- A crucial concept when writing an application able to send/receive network traffic is the ***socket***

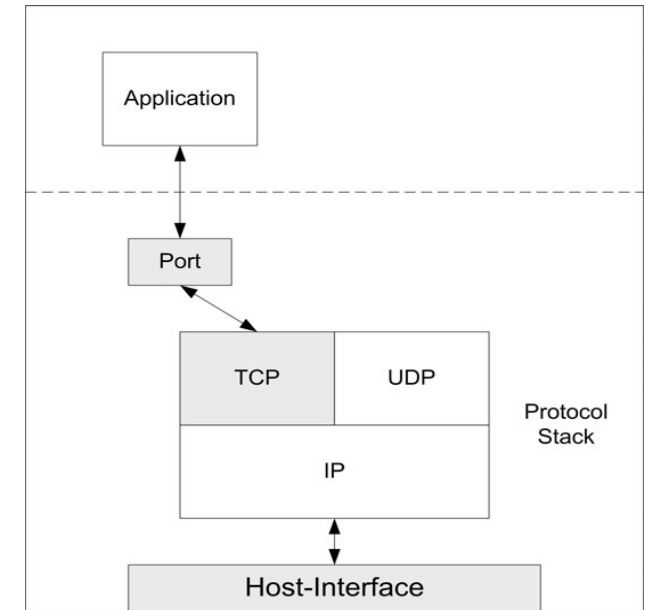
Sockets

- A socket is an **endpoint** of a communication channel between two **applications**
- Sockets enable the communication between two applications through the network, with a given communication protocol (i.e., set of common communication rules)
- Primitive of the **Operating System** enabling a program to access the network stack



Sockets host-protocol-port parameters

- When a new socket is created, it is important to define:
 - The **host/interface** through which messages should be sent/received
(**host = source or destination device/interface**)
 - The **protocol** to be used
(**protocol = set of rules that the communication should follow**)
 - The **port** (this is a concept mainly for IP-based protocols, but also for other network stacks such as the one used in vehicular networks) (**port = identifier of the source or destination application**)



Hosts in IPv4 sockets

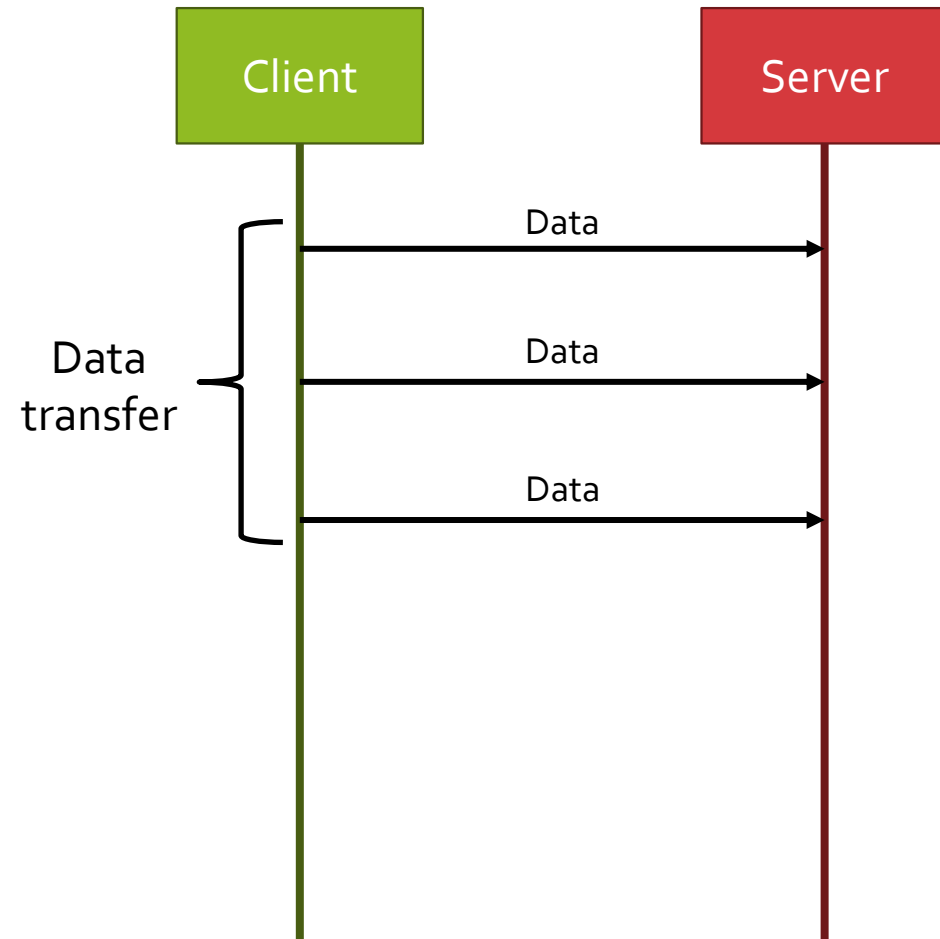
- Hosts are identified by addresses, and for IPv4 these are called **IP addresses**
- An **IPv4 address** (of the version 4 class) is defined as a 32-bit address, which is represented by four 8-bit values
- Example: 192.168.1.1
- By setting the host, it is possible to force a socket to use a specific **interface** (e.g., wlp3so, etho, ...)

Protocols in IPv4 sockets

- The protocol specifies the **details of communication over the socket**
- The two most common and used (transport) protocols are the Transmission Control Protocol (TCP) and the User Datagram Protocol (UDP)
- TCP is a stream-based reliable protocol, and UDP is a datagram (message)-based protocol that can be unreliable

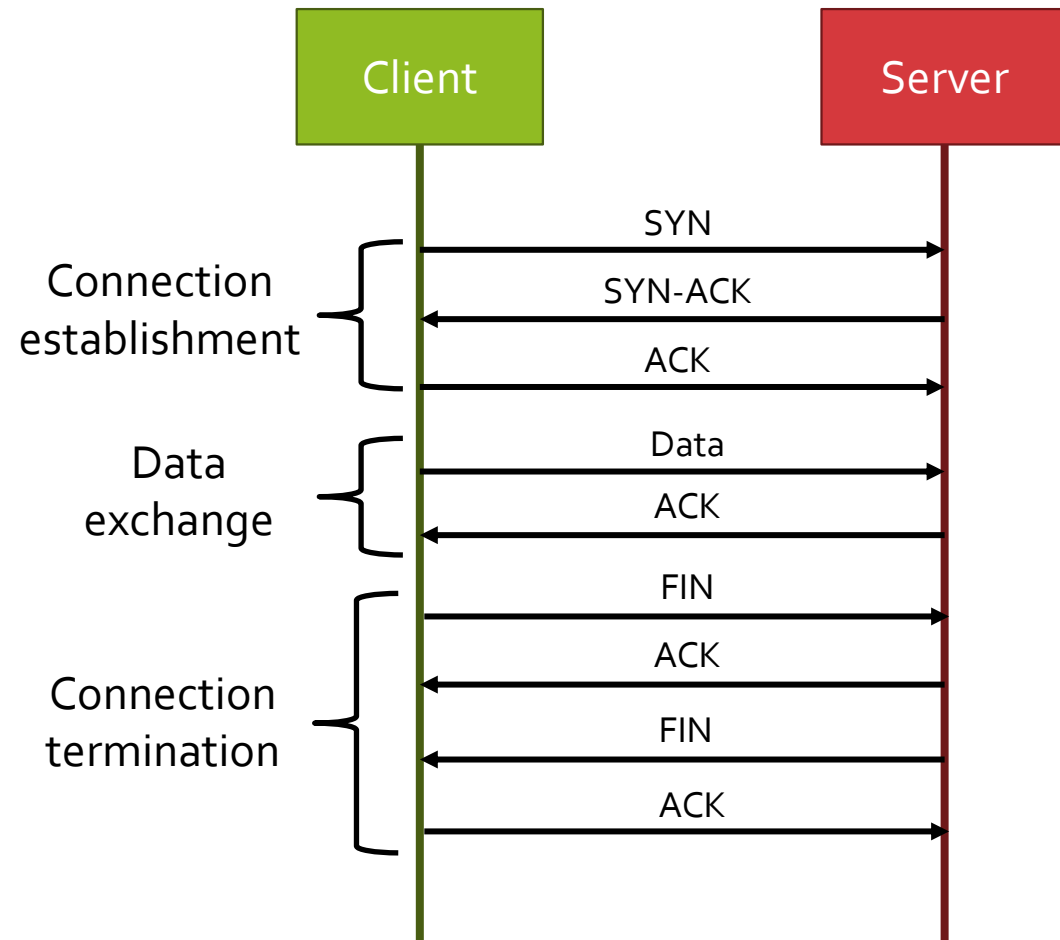
UDP

- **Connectionless protocol:** no handshake, no concept of connection (packets handled separately)
- **"Best effort"** service: packets may be lost or received out of order
- **No initial setup** is needed (more suitable for real-time multimedia in sensor networks)
- **Lightweight header:** src port, dst port, length, checksum



TCP

- **Connection-oriented protocol:** initial handshake (exchange of control messages), creating **connections** enabling bi-directional data flow
- Use of **acknowledgments** (ACKs)
- Performs congestion control and manages **retransmissions**
- **More extended header:** src port, dst port, seq. number, ACK number, flags, ...

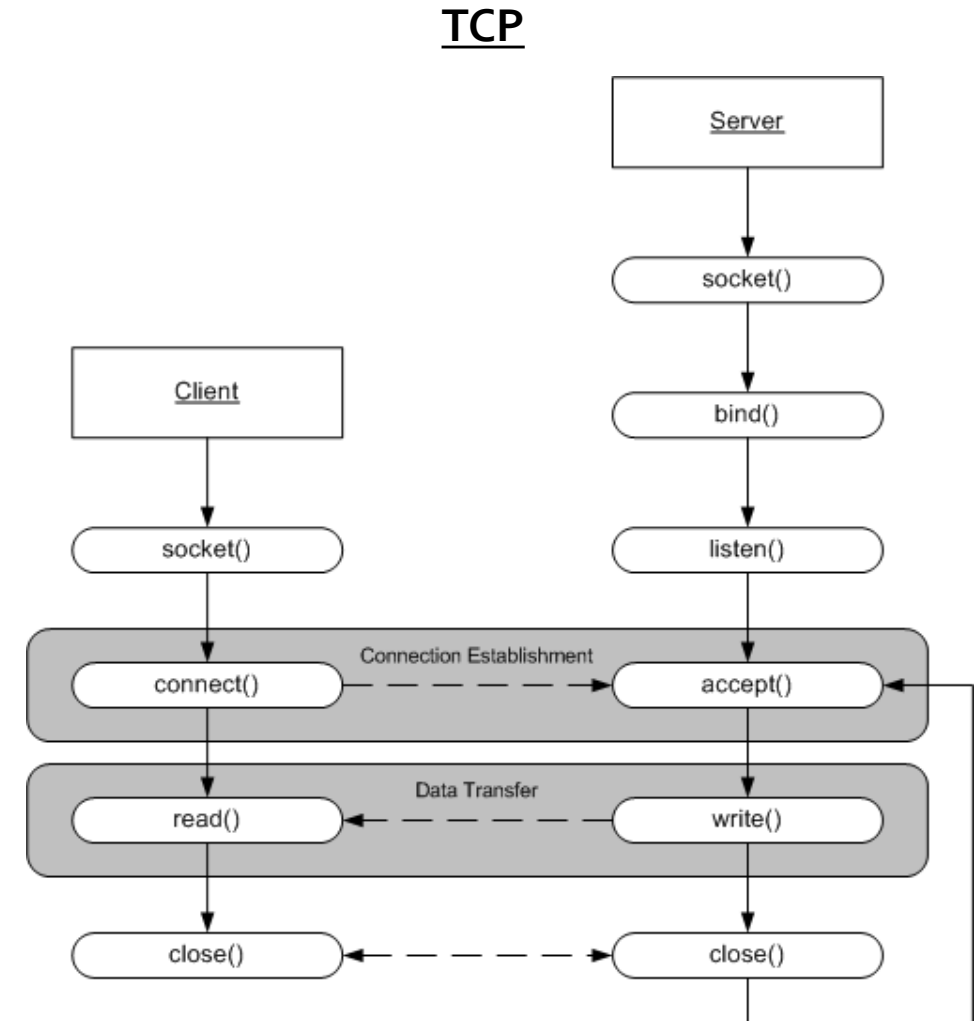


Ports in IPv4 sockets

- The **port** is the virtual point where network connections start and end
→ **number** assigned to uniquely identify a connection endpoint
- This is the application interface to the socket interface, and **identifies the application/service** → used to identify the destination application when sending packets
- Ports are unique on a host for a given protocol
- Ports are commonly called “**bound**” when they are attached to a given **socket**
- For IP-based protocols:
 - Ports are numbers that are split basically into two ranges
 - Port numbers below 1024 are reserved for *predefined services*
 - Port numbers above 1024 are typically used by applications (= you should use these ports for your custom applications)

Client/server model

- In most applications using sockets, a client-server model is used
- **Server:** responds to requests and possibly provides responses
- **Client:** makes requests/sends information to the server
- The Python sockets API (i.e., functions) provides commands that are specific to clients and to servers

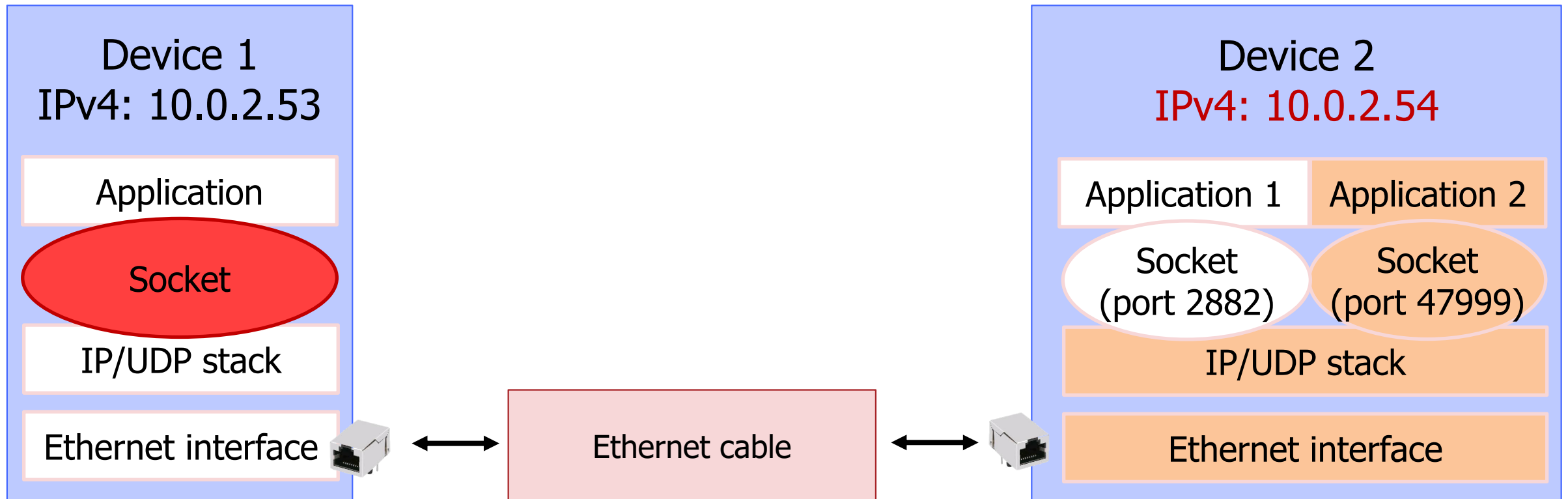


Client/server model - ports

- When using TCP/UDP, ports are divided into:
 - **Source ports:** used to identify the application that is the **source** of the data
 - **Destination ports:** used to identify the **destination** application that should receive the data
- When settings ports:
 - On the client: you specify the **destination port**, to which the server is listening; source port is usually selected **randomly** by the Operating System
 - On the server: you specify the **port to use for receiving packets**; if the server will need to reply to the client, it will use that port as source port, and as destination port the source port that was selected by the client

Python UDP socket example

- The application on device 1 should send data to **Application 2** on **Device 2**, that has opened a socket (communication endpoint) to wait for **UDP packets** on port 47999 (while another application waits for packets on another port)



Python UDP socket example: sender (client)

```
import socket
```

It imports the socket package

```
UDP_IP = "10.0.2.54"
```

It specifies the IP address of the **destination device**

```
UDP_PORT = 47999
```

It specifies the **destination** port for the packets → identifies the destination application (destination socket)

```
MESSAGE = "Hello, MSN!"
```

Content of the packet → simple string, in this example

```
print("UDP target IP: %s" % UDP_IP)
```

```
print("UDP target port: %s" % UDP_PORT)
```

```
print("message: %s" % MESSAGE)
```

```
sock = socket.socket(socket.AF_INET, # IPv4  
                     socket.SOCK_DGRAM) # UDP
```

Create a new IPv4 UDP socket (DGRAM) – TCP sockets are created with SOCK_STREAM

```
sock.sendto(MESSAGE.encode(), (UDP_IP, UDP_PORT))
```

Send packets to the server with IP address «10.0.2.54», using port 47999

The packet should be of **bytes** type; a string can be encoded into bytes with `encode()`

Python UDP socket example: receiver (server)

```
import socket
```

It imports the socket package

```
UDP_IP = "10.0.2.54"
```

It specifies the IP address of the interface where to **receive** data from

```
UDP_PORT = 47999
```

It specifies the port to use for **receiving** packets (packets with another *dest. port* won't be received)

```
sock = socket.socket(socket.AF_INET, # IPv4  
                     socket.SOCK_DGRAM) # UDP
```

Create a new IPv4 UDP socket (DGRAM) – TCP sockets are created with SOCK_STREAM

```
sock.bind((UDP_IP, UDP_PORT))
```

Bind socket to the listening interface address and port → listening for all packets destined to port 47999

```
while True:  
    data, addr = sock.recvfrom(1024)  
    print("received message: %s" % data.decode())
```

Receive and print all UDP packets with port 47999; we do this in an infinite loop to be able to receive more than one packet; 1024 B is the buffer size

The content of the packet is of **bytes** type; to be read as a string it should be decoded with `decode()`

Python UDP socket example

```
phd@phd-HP-EliteBook-830-16GB:~/MSN_0020$ python3 ./UDP_example_tx.py
UDP target IP: 10.0.2.54
UDP target port: 47999
message: Hello, MSN!
phd@phd-HP-EliteBook-830-16GB:~/MSN_0020$
```

```
phd-acer@phdacer-TravelMate-P214-52:~/MSN_0020$ python3 ./UDP_example_rx.py
received message: Hello, MSN!
```



985 4.005339919 10.0.2.53 10.0.2.54 UDP 60 58667 → 47999 Len=11

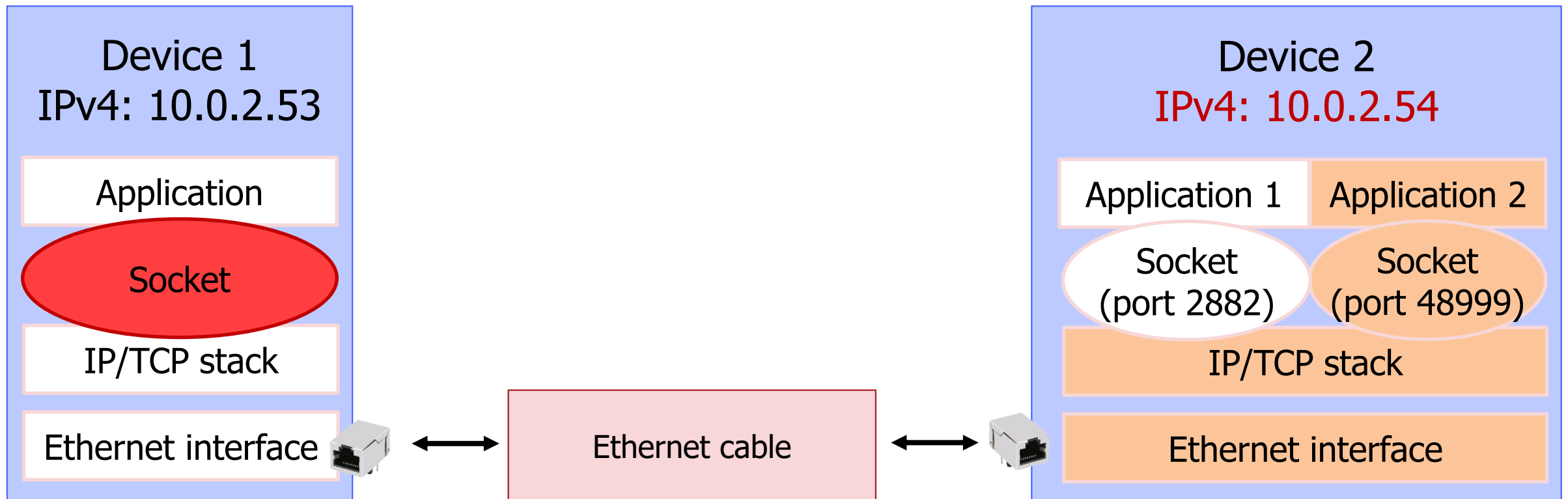
Header Checksum: 0x4932 [validation disabled]
[Header checksum status: Unverified]
Source Address: 10.0.2.53
Destination Address: 10.0.2.54
User Datagram Protocol, Src Port: 58667, Dst Port: 47999
Source Port: 58667
Destination Port: 47999
Length: 19
Checksum: 0x8e18 [unverified]
[Checksum Status: Unverified]
[Stream index: 3]
[Timestamps]
UDP payload (11 bytes)
Data (11 bytes)
Data: 48656c6c6f2c204d534e21

00 68 eb e0 84 77 08 00 45 00 ...o..h..w..E.
40 11 49 32 0a 00 02 35 0a 00 ...')@.@.I2...5..
00 13 8e 18 48 65 6c 6c 6f 2c ..6+....Hello,
00 00 00 00 00 00 ..MSN!....

"Hello, MSN!"

Python TCP socket example

- The application on device 1 should send a packet to **Application 2** on **Device 2**, that has opened a socket (communication endpoint) to wait for **TCP packets** on port 48999; when the packet is received, Application 2 should send back a message containing "OK" to the application on device 1



Python TCP socket example: client

```
import socket

TCP_IP = "10.0.2.54"
TCP_PORT = 48999
MESSAGE = "Hello, MSN!"

s = socket.socket(socket.AF_INET, # IPv4
                  socket.SOCK_STREAM) # TCP
s.connect((TCP_IP, TCP_PORT))

s.send(MESSAGE.encode())

data = s.recv(1024)
print(data.decode())

s.close()
```

TCP connections must be closed
when no more used

Difference w.r.t. UDP → we create here a TCP (SOCK_STREAM) socket

Sending packets is managed differently as TCP is a stream-based protocol. First, we need to connect to a remote socket of a remote application (performing the 3-way handshake)

We use send() instead of sendto() as we are already connected to a single remote socket

As TCP is a stream-based protocol, we can easily wait for the reply from the server using recv(), that returns just the received packet, as opposed to recvfrom()

Python TCP socket example: server

```
import socket

TCP_IP = "10.0.2.54"
TCP_PORT = 48999

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind((TCP_IP, TCP_PORT))
s.listen(1)

conn, addr = s.accept()
print("Connection address:", addr)
while True:
    data = conn.recv(1024)
    if not data: break
    print("received message: %s" % data.decode())
    reply = "OK"
    conn.send(reply.encode())
conn.close()
```

Difference w.r.t. UDP → after binding, we must now listen for incoming connections; we accept here 1 connection at a time

After the socket is bound and listening, `accept()` will wait for connections and block until a connection is established

data will be "None" if a reception error occurs, or **to signal a disconnection** from the client

As TCP is a stream-based protocol, we can easily reply to the client that just connected with `send()`

N.B.: *send* and *recv* operations are performed on **conn**, not on **s**!

Python TCP socket example

```
phd@phd-HP-EliteBook-830-16GB:~/MSN_0020$ python3 ./TCP_
example_tx.py
OK
phd@phd-HP-EliteBook-830-16GB:~/MSN_0020$ |
```

```
phd-acer@phdacer-TravelMate-P214-52:~/MSN_0020$ python3
./TCP_example_rx.py
Connection address: ('10.0.2.53', 60166)
received message: Hello, MSN!
```



Client



Server



756	3.437069436	10.0.2.53	10.0.2.54	TCP	74	39956 → 48999	[SYN, Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM T...
757	3.437093614	10.0.2.54	10.0.2.53	TCP	74	48999 → 39956	[SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 ...
758	3.437623361	10.0.2.53	10.0.2.54	TCP	66	39956 → 48999	[ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=25835579...
759	3.437623626	10.0.2.53	10.0.2.54	TCP	77	39956 → 48999	[PSH, ACK] Seq=1 Ack=1 Win=64256 Len=11 TSval=25...
760	3.437661099	10.0.2.54	10.0.2.53	TCP	66	48999 → 39956	[ACK] Seq=1 Ack=12 Win=65152 Len=0 TSval=2319367...
761	3.437877001	10.0.2.54	10.0.2.53	TCP	68	48999 → 39956	[PSH, ACK] Seq=1 Ack=12 Win=65152 Len=2 TSval=23...
763	3.438558546	10.0.2.53	10.0.2.54	TCP	66	39956 → 48999	[ACK] Seq=12 Ack=3 Win=64256 Len=0 TSval=2583557...
764	3.438558639	10.0.2.53	10.0.2.54	TCP	66	39956 → 48999	[FIN, ACK] Seq=12 Ack=3 Win=64256 Len=0 TSval=25...
765	3.438658771	10.0.2.54	10.0.2.53	TCP	66	48999 → 39956	[FIN, ACK] Seq=3 Ack=13 Win=65152 Len=0 TSval=23...
766	3.438783957	10.0.2.53	10.0.2.54	TCP	66	39956 → 48999	[ACK] Seq=13 Ack=4 Win=64256 Len=0 TSval=2583557...

Connection establishment
"Hello, MSN!"
"OK"
Connection termination

```
▼ Ethernet II, Src: HP_e0:84:77 (00:68:eb:e0:84:77), Dst: QuantaComput_f0:6f:95 (b4:a9:fc:f0:6f:95)
  ► Destination: QuantaComput_f0:6f:95 (b4:a9:fc:f0:6f:95)
  ► Source: HP_e0:84:77 (00:68:eb:e0:84:77)
  Type: IPv4 (0x0800)
▼ Internet Protocol Version 4, Src: 10.0.2.53, Dst: 10.0.2.54
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  ► Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 63
  Identification: 0x6b52 (27474)
  ► 010. .... = Flags: 0x2, Don't fragment
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 64
  Protocol: TCP (6)
  Header Checksum: 0xb6fc [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 10.0.2.53
  Destination Address: 10.0.2.54
  ► Transmission Control Protocol, Src Port: 39956, Dst Port: 48999, Seq: 1, Len: 11
  ▼ Data (11 bytes)
    Data: 48656c6c662c204d534e21
```

```
b4 a9 fc f0 6f 95 00 68 eb e0 84 77 08 00 45 00 ...o..h...w..E..
00 3f 6b 52 40 00 40 06 b6 fc 0a 00 02 35 0a 00 ...?kR@.@.....5..
02 36 9c 14 bf 67 e7 56 b6 e4 53 ee b9 e8 80 18 ...6...g.V...S...
01 f6 c2 ee 00 00 01 01 08 0a 99 fd f7 4d 8a 3e .....M>
bd a8 48 65 6c 6c 6f 2c 20 4d 53 4e 21 ...Hello, MSN!
```